# Digital Systems

## Lecture 11

## The Interrupt Mechanism

Mark van der Wilk

Hilary Term 2026

University of Oxford
Department of Computer Science

Following the course of past years by Mike Spivey

# Last Lecture...

We saw:

- How to program with interrupts
  - 1. Configure hardware to send interrupts, 2. write handler.
- How interrupts know which function to call
  - `__vectors` table, with addresses to handlers, in expected locations

## ? What *actually* happens on an interrupt?

- Last lecture, we showed how to program an interrupt in a high-level language.
- … this course is about you being able to understand every detail so you can *actually build* a computer.
- So we'll need to dive deeper:
  - What does hardware actually do on an interrupt?
  - What does the hardware do to the *state* of the CPU on an interrupt?

# What additional tasks must subroutines do?

- An interrupt looks like a subroutine, inserted at an arbitrary point.
- Interrupt *handlers* can be normal subroutines.

```
int prime(int n) {
    int k = 2;

    while (k * k <= n) {
        if (n % k == 0) return 0;
        poll_uart();
        k++;
    }

    return 1;
}
```

```
void poll_uart(void) {
    if (UART_TXDRDY) {
        // send another char
    }
}
```

# Satisfying the Subroutine Convention in Interrupts

**? What *additional* work needs to be done on an interrupt, compared to calling a normal subroutine?**

- Remember the subroutine convention.
  What does the interrupt handler expect?
- What is different about an interrupt cf a normal subroutine?
  What does the interrupted code assume?

**⚠ Parent's code does not know it will be interrupted!**

So we need to more careful with storing/restoring *state*.

- Must also store `psr`, `pc`, `lr`, `r12`, `r0 - r3`!

# Satisfying the Subroutine Convention in Interrupts

```
int prime(int n) {
    int k = 2;

    while (k * k <= n) {
        if (n % k == 0) return 0;
        k = ... k+1;
    }

    return 1;
}
```

Interrupts must save the processor state

```
void uart_handler(void) {
    if (UART_TXDRDY) {
        // send another char
    }
}
```

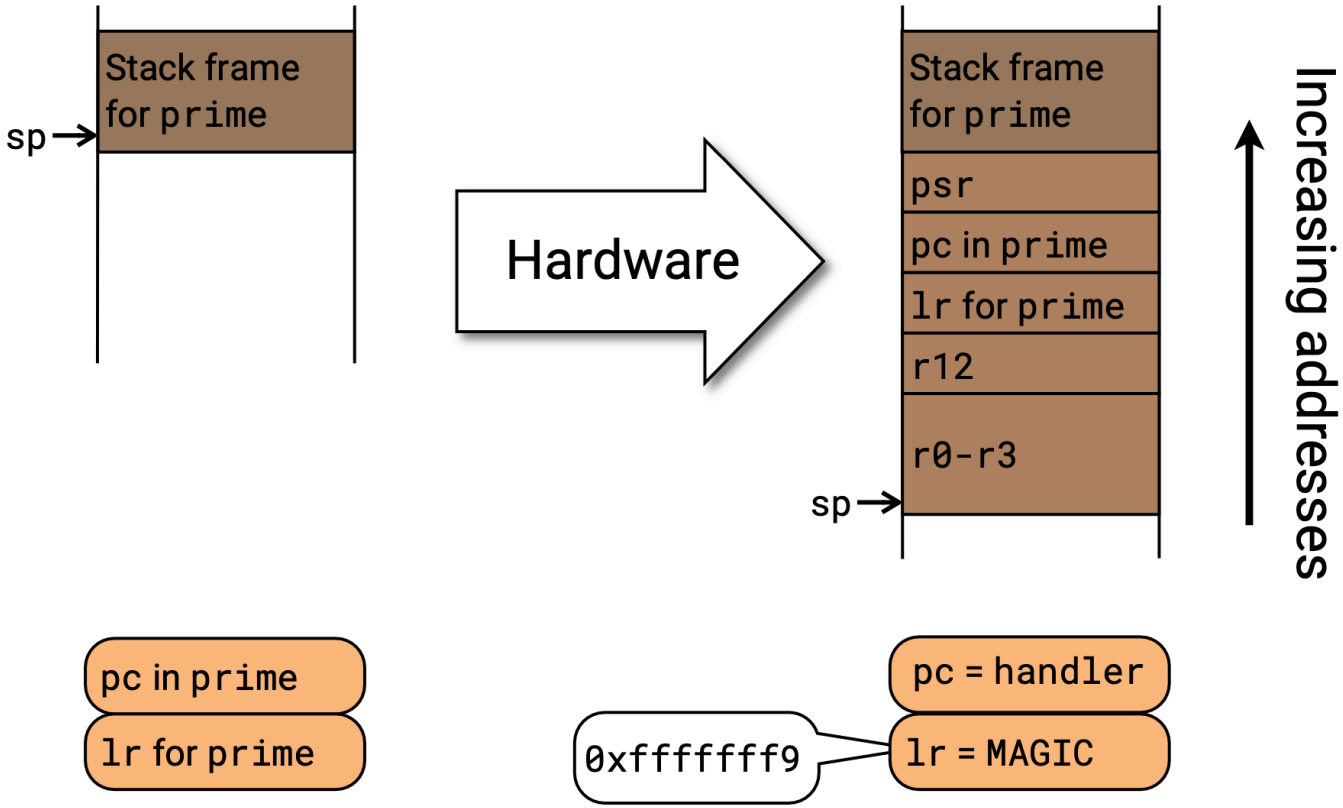
Interrupt handlers can be ordinary subroutines

# Satisfying the Subroutine Convention in Interrupts

- Must restore `psr` on returning from the interrupt.
  Code doesn't know that `psr` can be overwritten, as for normal subroutine calls.
- Must satisfy rest of calling convention (e.g. handler can use `r0 - r3`).

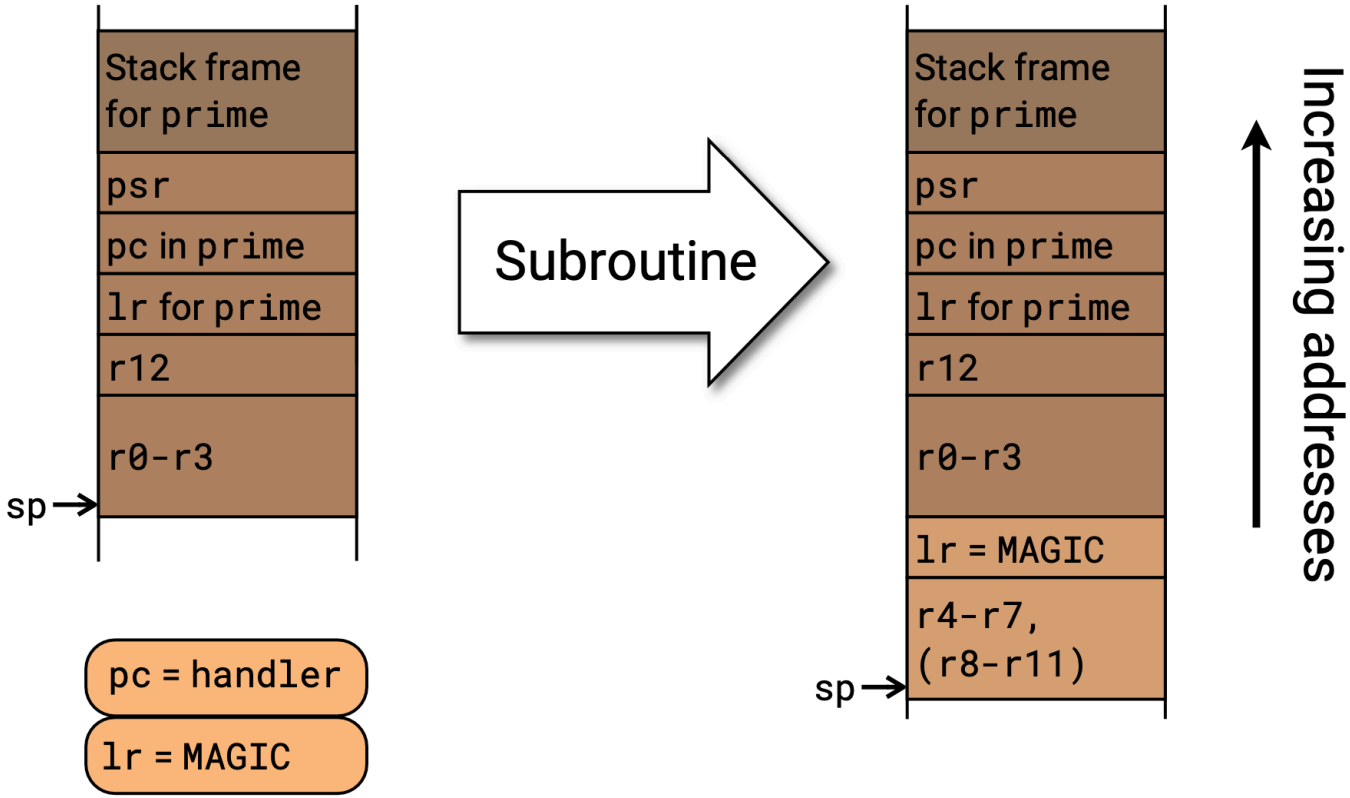
# Hardware Interrupt Process

Exception / interrupt frame. State is same as after normal subroutine call.



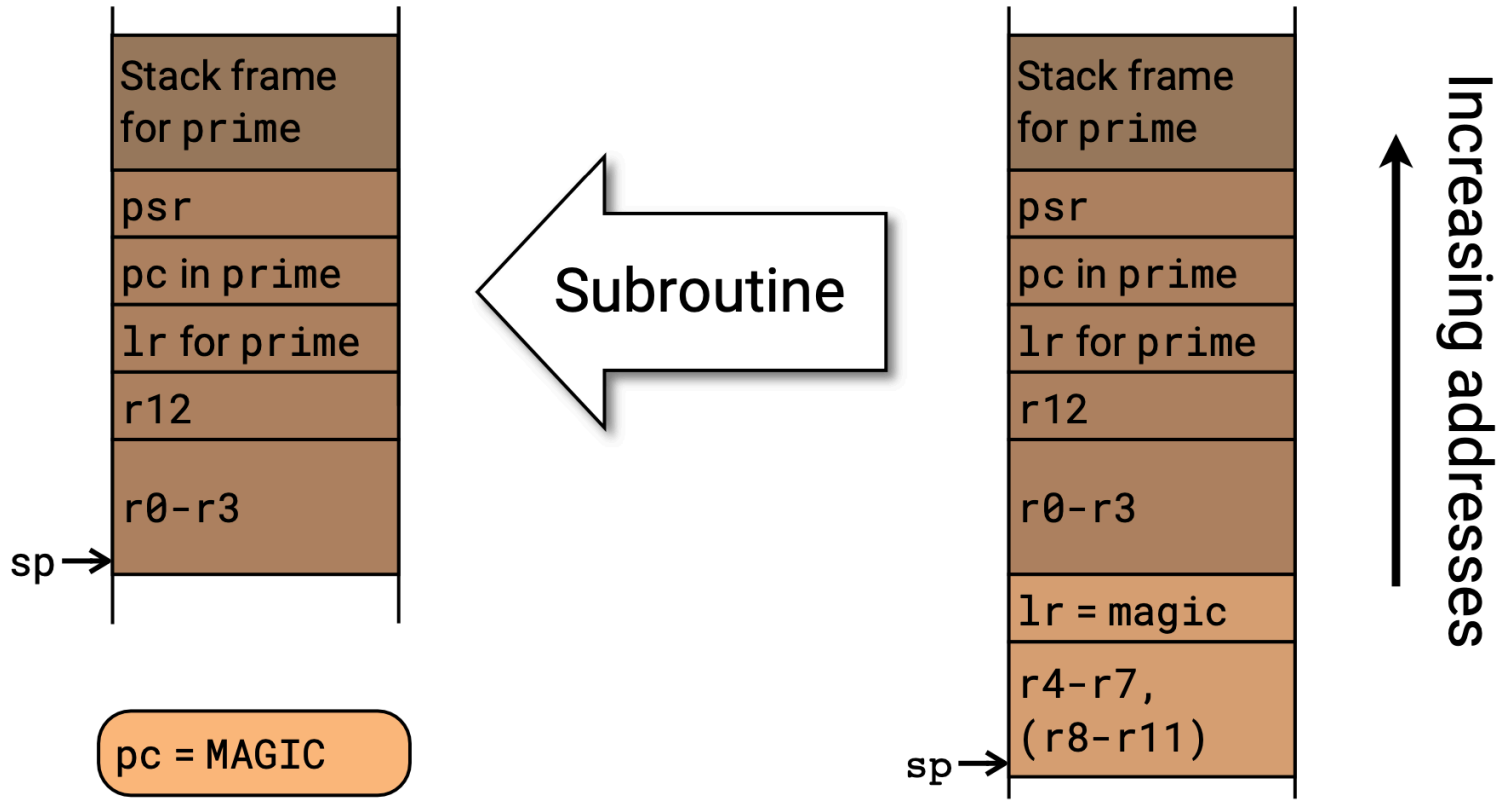
# Entering Handler (that can call subroutines)

Can push additional registers in the usual way.



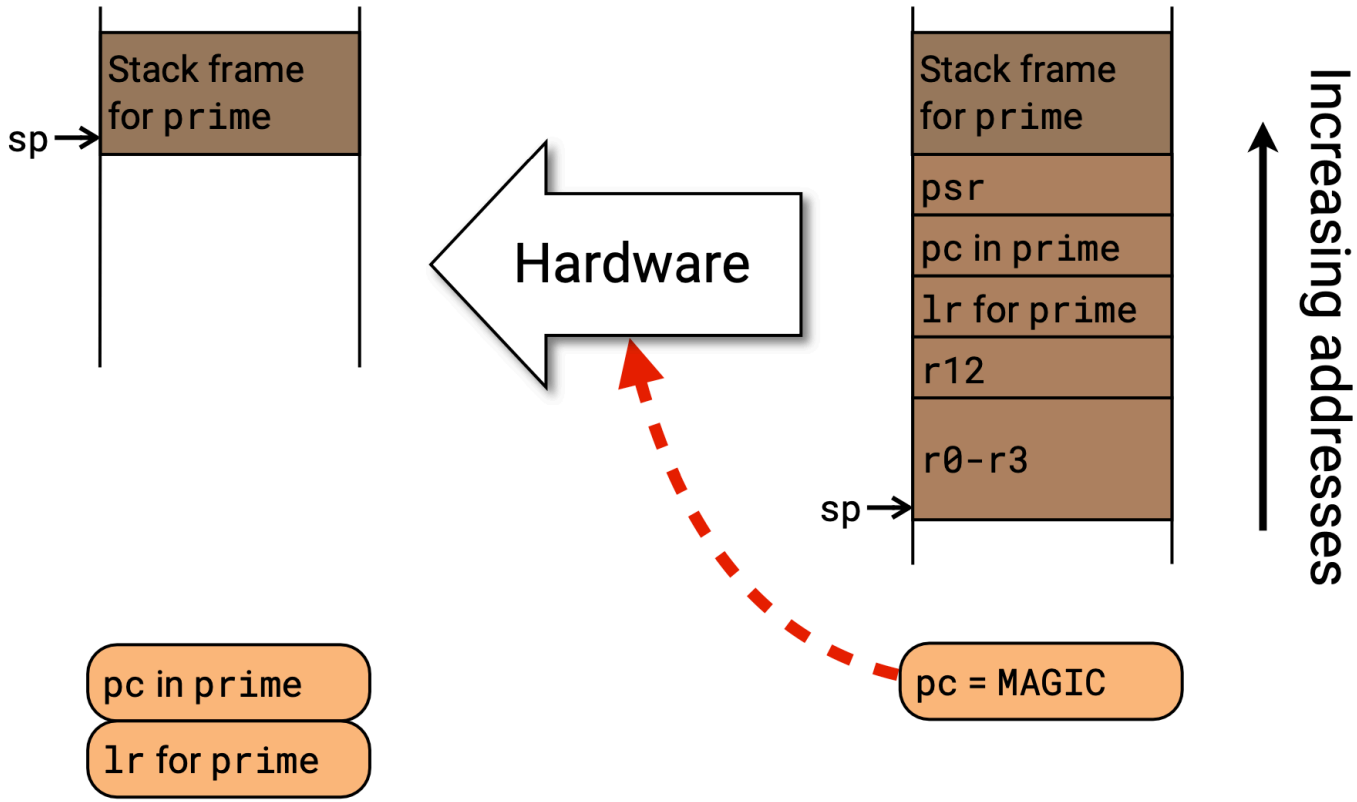
# Exiting Handler (what software must do)

Can return from subroutine in usual way (`bx lr` or `pop`).



# Exiting Interrupt

pc is set to magic number ⇒ follow return from interrupt procedure.



# Hardware Obeys Calling Conventions

Advantages

- Interrupt handlers can be ordinary subroutines.
- No need for assembly-code adapters.

Disadvantages

- Interrupt latency is fixed and large.

# Example: Timers

# Scheduling Regular Actions

Version 0: delay loops (already seen).

- Wasteful of time and power.

Version 1: use a timer for delays (this lecture).

- Easier to maintain, power efficient, but still wasteful of CPU time.

Version 2: purely interrupt driven (this lecture).

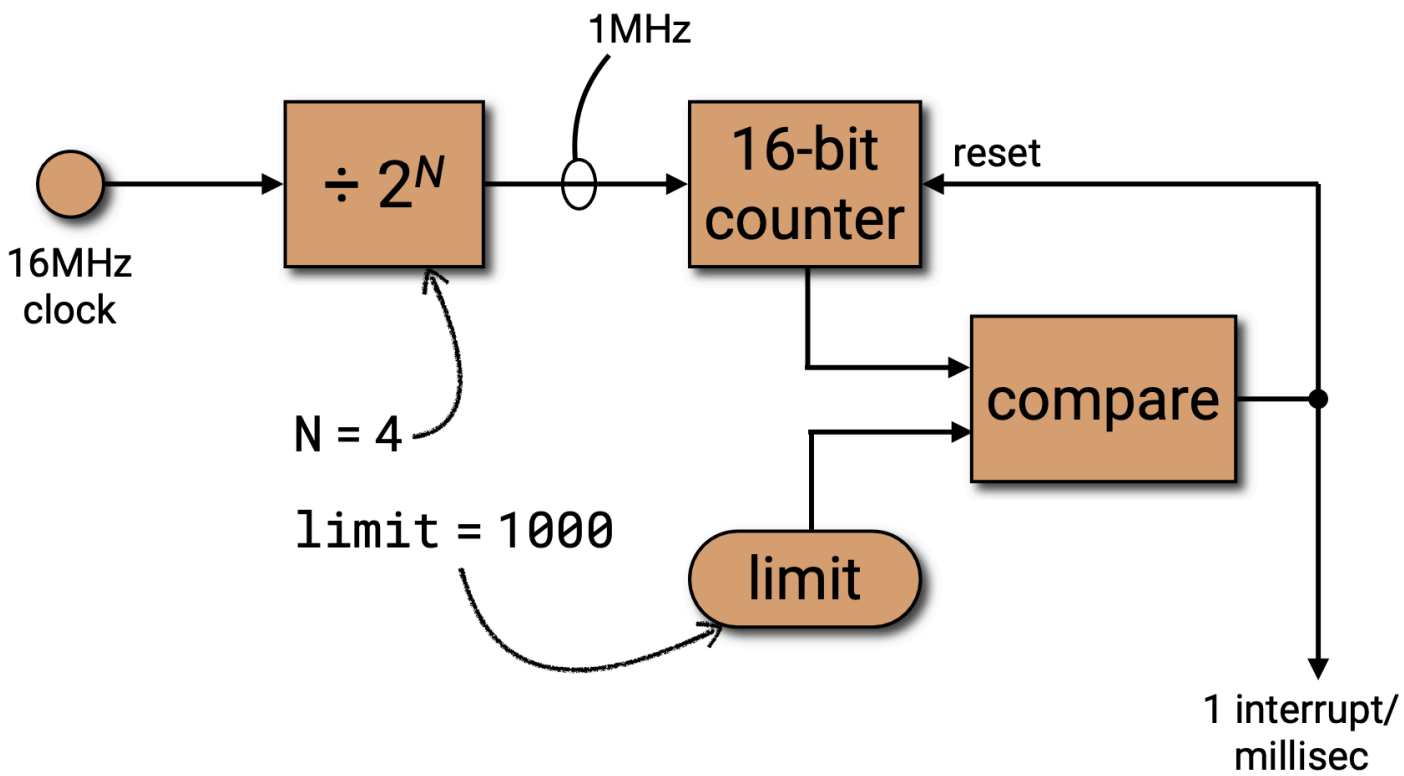
- Allows computation while waiting but inflexible.

Version 3: use an operating system (next time).

- Best of all worlds!

# Timers

Multiple compare circuits and counters (up to 32-bit) are available.



# Delay v1: Power Efficient

```
unsigned volatile millis = 0;
void timer1_handler(void) {
    if (TIMER1_COMPARE[0]) {
        millis++;
        TIMER1_COMPARE[0] = 0;
    }
}
void delay(unsigned usec) {
    // Interface gets µs, we offer ms resolution...
    unsigned goal = millis + usec/1000;
    while (millis < goal) { pause(); } // Uses wfe instruction
}
```

# Aside...

```
void delay(unsigned usec) {
    unsigned goal = millis + usec/1000;
    while (millis < goal) { pause(); } // Uses wfe instruction
}
```

- If `goal` overflows, loop terminates immediately ⇒ maximum delay gets smaller as `millis` increases! How to fix?
- `while ((millis - start_millis) < usec / 1000)`
- *Still works* if `millis` overflows!

## Delay v2: Interrupt-driven

To allow computation while waiting, make `timer_interrupt` call this at 5ms intervals:

```
static int row = 0;
void advance(void) {
    row++;
    if (row == 3) row = 0;
    GPIO_OUT = heart[row];
}
```

- Compare to `delay()` solution where beating heart is shown with loop.
- Here: no internal control structure allowed.
- Efficient but inflexible.

## Summary

- Details of how interrupts work.
- Timer circuits.

**? How can we allow multiple subroutines to operate in an interleaved way?**

- Currently, we can have one "main" function, that is interrupted. This function is special.
- How can we have multiple processes running concurrently, that are interrupted to interleave with one another?